

FLOOD RESILIENCE & RESPONSE PLATFORM

Technical Project Submission

Project Purpose

A web-based disaster-management platform connecting flood preparedness, community reporting, drone-based situational awareness, resource coordination, hazard-aware routing and shelter tracking.

Core Focus: Response Coordination Engine

The platform is designed around a human-in-the-loop response workflow. Information from citizens and simulated drone observations is converted into an operational state, which is then used to recommend resources, volunteers and routes for authority approval.

Submission Scope: Technical proposal / MVP specification. Where implementation details are not yet completed, they are described as proposed rather than claimed as deployed.

Table of Contents

- 1. Executive Summary & Core Value Proposition
 - 1.1 Executive Summary
 - 1.2 System Vision, Scope & Primary Objectives
 - 1.3 Key Differentiators & Industry Alignment
- 2. System Architecture & Technical Design
 - 2.1 Overall System Architecture & Data Flow
 - 2.2 Comprehensive Technology Stack
 - 2.3 Modular System Components
 - 2.4 Infrastructure & Scalability Strategy
- 3. Repository & Directory Structure
 - 3.1 Folder Hierarchy & File Layout
 - 3.2 Key File Responsibilities
 - 3.3 Codebase Navigation Guide
 - 3.4 Current GitHub Repository Snapshot
- 4. Detailed Feature Breakdown & User Workflows
 - 4.1 Security & Credential Management
 - 4.2 Core Business Logic & Execution Engines
 - 4.3 User Interface & Command Dashboard
 - 4.4 Real-Time Systems & Integration
- 5. Mathematical Foundations & Quantitative Frameworks
 - 5.1 Zone Priority Model
 - 5.2 Resource Allocation Model
 - 5.3 Volunteer Matching Model
 - 5.4 Route Hazard Model
 - 5.5 Mathematical Summary Matrix
- 6. Database Schema & Security Infrastructure
 - 6.1 Entity-Relationship Overview
 - 6.2 Database Tables
 - 6.3 Security & Row-Level Access
- 7. Local Development & Installation Guide

7.1 Prerequisites

7.2 Setup

7.3 Environment Variables

7.4 Running the Development Suite

8. Verification, Testing & Quality Assurance

8.1 Static Verification

8.2 Feature Validation

8.3 Edge Cases & Guardrails

9. Multi-Platform Deployment & DevOps

9.1 Web Deployment

9.2 Containerization

9.3 Optional Mobile Packaging

10. Changelog, Roadmap, Troubleshooting & License

10.1 Development History

10.2 Roadmap

10.3 Troubleshooting

10.4 Licensing & Intellectual Property

10.5 Error Log Book

Appendix A — Core Technology Reference

Appendix B — Core Coordination Pseudocode

Appendix C — Submission Integrity

Appendix D — Executable Database DDL & RLS Reference

Appendix E — Deployment Configuration Reference

Appendix F — Environment & Secret Reference

Appendix G — License Text

Appendix H — Submission Verification Matrix

1. Executive Summary & Core Value Proposition

1.1 Executive Summary

Flood disasters create a coordination problem as much as a physical hazard problem. During a flood, information arrives from multiple sources, roads can become inaccessible, available resources change, citizens may be separated from family members, and responders must act using incomplete and changing information.

The proposed Flood Resilience & Response Platform addresses this problem through a connected operational workflow rather than a collection of independent tools. The platform combines a browser-based WebXR preparedness simulation, drone-based situational awareness, community reporting, geospatial state management, response coordination, route validation and shelter check-in.

The central component is the **Response Coordination Engine**. It receives the current state of affected zones and evaluates available resources and volunteers based on suitability, availability, capability, skill and distance. It then produces a recommendation for a human authority to review.

The platform deliberately uses a human-in-the-loop model. Computer vision is used to detect people and vehicles from simulated or pre-recorded drone footage. It is not presented as a reliable property-damage detector. Road and infrastructure damage can instead be reported by people in the affected area.

1.2 System Vision, Scope & Primary Objectives

The system vision is to convert fragmented flood information into a shared operational picture and then convert that picture into an actionable, reviewable response recommendation.

Objective	Engineering interpretation
O1 — Preparedness	Provide a browser-based WebXR scenario that teaches users how rising water levels can affect evacuation and rescue decisions.
O2 — Situation Awareness	Combine drone observations and community reports into a common geospatial operational state.
O3 — Resource Coordination	Prioritize affected zones and recommend appropriate resources based on capability, availability and distance.
O4 — Volunteer Coordination	Match available citizens/volunteers to suitable response tasks based on their profile and skills.
O5 — Route Safety	Generate candidate routes and check them against known flood or road hazards.
O6 — Rescue Tracking	Record shelter check-ins so the system can represent a rescued person's safe status.

1.3 Key Differentiators & Industry Alignment

The differentiation of the proposed platform is not the use of WebXR, drones, maps or machine learning individually. The differentiation comes from connecting those capabilities into a response workflow.

1. Preparedness-to-response continuity. The same platform covers both learning before a disaster and coordination during a disaster.

2. Human-plus-community intelligence. Drone observations provide one source of information while citizens can directly report conditions that computer vision may not reliably infer.

3. Coordination as the central intelligence layer. The system does not stop at displaying information. It uses the information to recommend who and what should respond.

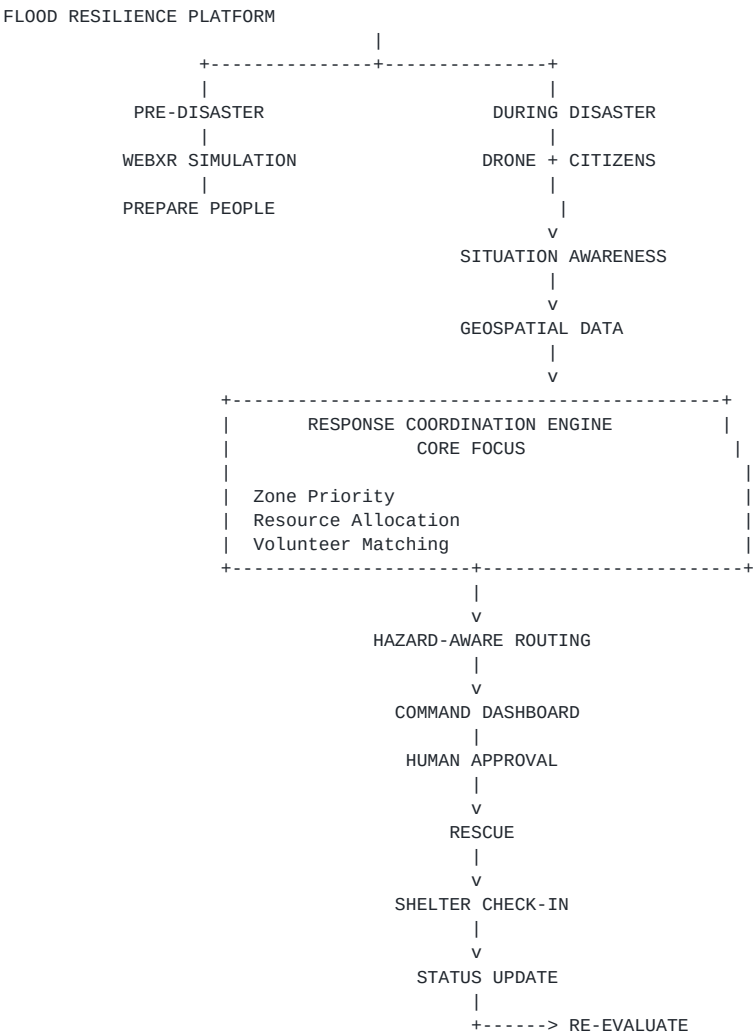
4. Human approval. Automated scoring does not directly dispatch rescue teams. Recommendations remain reviewable by an authorized operator.

2. System Architecture & Technical Design

2.1 Overall System Architecture & Data Flow

The architecture follows the operational sequence: **Prepare** → **Sense** → **Analyze** → **Coordinate** → **Route** → **Rescue** → **Track** → **Re-evaluate**.

The preparedness phase uses WebXR to simulate flood situations. During an actual or simulated disaster, drone observations and citizen reports contribute information to the shared geospatial state. The coordination engine uses this state to produce response recommendations.



Core architectural principle: computer vision provides observations; the coordination engine generates recommendations; the command dashboard provides human oversight before a response action is approved.

2.2 Comprehensive Technology Stack

Layer	Technology	Purpose
Frontend	React	Web application and dashboards.
Maps	Mapbox GL JS	Map, zones, reports, shelters and routes.
Simulation	A-Frame / Three.js + WebXR	Browser-based flood preparedness simulation.
Offline Web	PWA + IndexedDB	Installable application and pending-report storage.

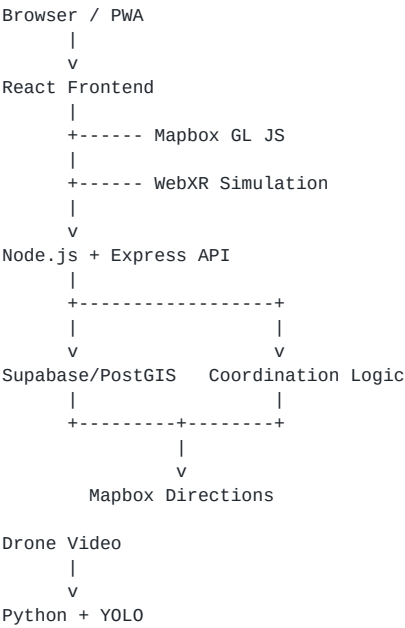
Layer	Technology	Purpose
Backend	Node.js + Express	API layer and business logic.
AI / CV	Python + Ultralytics YOLOv8	People and vehicle detection from drone footage.
Database	Supabase + PostgreSQL + PostGIS	Persistent and geospatial application data.
Routing	Mapbox Directions API	Candidate route generation.
Spatial Analysis	Turf.js	Route and hazard geometry operations.
Deployment	Docker	Reproducible packaging and deployment.

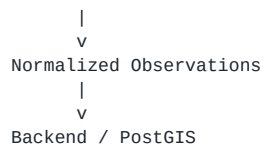
2.3 Modular System Components

Module	Input / Trigger	Processing	Output
WebXR Simulation	Scenario selection	Water-level and scenario progression	Training and decision feedback
Drone Intelligence	Drone footage	Frame processing + YOLO inference	People/vehicle observations
Community Reporting	Citizen report	Validation + geospatial storage	Hazard/incident report
Geospatial Data	Validated observations	Persistence + spatial queries	Current operational state
Coordination Engine	Zones + resources + profiles	Priority + suitability scoring	Recommended assignment
Routing	Assignment + hazards	Route generation + hazard check	Candidate safer route
Command Dashboard	Current state + recommendation	Human review	Approve / modify / reject
Shelter Tracking	Rescue/check-in	Status update	Safe / checked-in status

2.4 Infrastructure & Scalability Strategy

The MVP is intentionally modular rather than unnecessarily distributed. The frontend, backend, AI processing and database have clear responsibilities while avoiding infrastructure that does not directly contribute to the demonstrated workflow.





Potential future scaling can include dedicated AI workers, job queues, database connection pooling, caching and more advanced offline capabilities. These are not required to demonstrate the MVP.

3. Repository & Directory Structure

3.1 Folder Hierarchy & File Layout

```
flood-resilience/
|
+-- frontend/
|   +-- src/
|   |   +-- components/
|   |   +-- pages/
|   |   +-- maps/
|   |   +-- simulation/
|   |   +-- reports/
|   |   +-- dashboard/
|   |   +-- services/
|   |   +-- types/
|   |   |-- main.tsx
|   +-- public/
|   |-- package.json
|
+-- backend/
|   +-- src/
|   |   +-- routes/
|   |   +-- controllers/
|   |   +-- services/
|   |   +-- coordination/
|   |   +-- routing/
|   |   +-- validation/
|   |   +-- database/
|   |   |-- server.ts
|   |-- package.json
|
+-- ai/
|   +-- detection/
|   |   +-- inference.py
|   |   |-- preprocessing.py
|   |-- requirements.txt
|
+-- database/
|   +-- migrations/
|   |-- seeds/
|
+-- tests/
|   +-- frontend/
|   +-- backend/
|   +-- coordination/
|   |-- routing/
|
+-- docker/
|   |-- Dockerfile
|
+-- docs/
|
|-- README.md
```

3.2 Key File Responsibilities

Directory / File	Responsibility
frontend/src/simulation	WebXR scenario loading, water-level state and simulation interaction.
frontend/src/maps	Map rendering, markers, hazard polygons and route display.
frontend/src/reports	Citizen reporting forms and submission state.
frontend/src/dashboard	Authority view of zones, resources and recommendations.
backend/src/coordination	Zone priority, resource allocation and volunteer matching.
backend/src/routing	Route requests and hazard-intersection workflow.
backend/src/validation	Input validation, permissions and report normalization.

Directory / File	Responsibility
ai/detection	Drone-frame preprocessing and YOLO inference.
database/migrations	Version-controlled schema changes.
tests	Automated verification and edge-case testing.

3.3 Codebase Navigation Guide

A developer debugging a citizen report should follow the path report UI → API route → validation → database persistence → geospatial state → dashboard.

A developer debugging resource allocation should follow zone state → coordination service → scoring functions → selected resource/volunteer → route service → dashboard approval.

A developer debugging drone detection should follow uploaded video → frame extraction → YOLO inference → normalized detection → API ingestion → geospatial record.

3.4 Current GitHub Repository Snapshot

The current repository baseline contains the PDF-generation prototype and submission artifacts. The application modules described in the target architecture are implementation scope and should be added incrementally.

```
flood-resilience-response-platform/  
|  
+-- main.py  
+-- requirements.txt  
+-- README.md  
+-- LICENSE  
+-- .gitignore  
+-- output/  
|   |-- Flood_Resilience_Submission.pdf  
|-- test.pdf
```

GitHub: github.com/harichan18/flood-resilience-response-platform

The repository baseline is version-controlled on the main branch. Future application commits should add frontend, backend, database, AI and test modules as they are implemented.

4. Detailed Feature Breakdown & User Workflows

4.1 Security & Credential Management

The platform separates citizen-facing actions from authority-level operational actions. Authentication and authorization should be handled through the selected Supabase authentication model or an equivalent backend-managed mechanism.

Actor	Typical Permissions
Citizen	Create reports/SOS and view permitted personal or public status.
Volunteer	Maintain profile, availability and receive permitted assignments.
Authority	View operational dashboard, manage resources and approve recommendations.
Shelter Operator	Record authorized shelter check-ins and occupancy.

4.2 Core Business Logic & Execution Engines

4.2.1 WebXR Flood Simulation

Input / Trigger: a learner selects a flood scenario and starts the simulation.

Processing: the scenario progresses through predefined water-level and situational states. The learner encounters decisions involving movement, evacuation and rescue response.

Output / Action: the interface provides feedback based on the selected action and helps the learner understand how different flood conditions affect decisions.

4.2.2 Drone-Based Situational Awareness

Input / Trigger: simulated or pre-recorded drone footage is provided to the prototype.

Processing: frames are extracted and YOLO is used to identify target classes such as people and vehicles.

Output / Action: detections become observations that can be represented on the operational map.

Scope limitation: property damage is not treated as a reliably solved YOLO task in the MVP. Road and infrastructure conditions can instead be supplied through community reports.

4.2.3 Community Reporting

Input / Trigger: a citizen reports an incident such as a blocked road, damaged infrastructure, rising water, stranded person or emergency request.

Processing: the report is validated, timestamped, geolocated and stored with its category, severity and optional evidence.

Output / Action: the report becomes part of the shared operational state and can influence zone priority or route safety.

4.2.4 Response Coordination Engine — CORE FOCUS

Input / Trigger: the system receives an updated operational state for one or more affected zones.

Processing: the engine calculates zone priority and evaluates available resources and volunteers according to capability, skill, availability, suitability and distance.

Output / Action: the engine produces a recommended resource and volunteer assignment for human review.

Key principle: the engine recommends a response. It does not independently authorize or perform the rescue.

4.2.5 Hazard-Aware Route Optimization

Input / Trigger: a resource needs to move from its current location toward an affected zone or shelter.

Processing: a candidate route is generated and checked against known hazard geometry such as blocked roads or severe flood zones.

Output / Action: a candidate safer route is shown to the authority. If unsuitable, an alternative route can be requested.

4.2.6 Shelter Check-In & Family Status

Input / Trigger: a rescued person reaches a registered shelter and is checked in.

Processing: the shelter record and person's status are updated while preserving access control.

Output / Action: authorized family members can see that the person has reached a safe shelter rather than relying only on informal communication.

4. Detailed Feature Breakdown & User Workflows — Continued

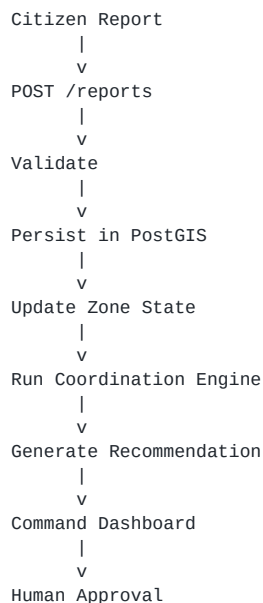
4.3 User Interface & Command Dashboard

The command dashboard is designed as an operational interface rather than a decorative visualization. Its purpose is to allow an authority to understand what is happening, why a recommendation was produced and what action can be approved.

Dashboard Area	Information Displayed
Zone Status	Priority, severity, affected count, active SOS and major hazards.
Resource Panel	Resource type, capability, availability and location.
Volunteer Panel	Relevant skills, availability and suitability.
Recommendation	Suggested assignment and reasoning.
Route	Candidate route and detected hazard conflicts.
Approval Controls	Approve, modify or reject recommendation.
Shelter Panel	Capacity, occupancy and check-in status.
Reports Layer	Citizen reports and verification state.

4.4 Real-Time Systems & Integration

The MVP does not claim a fully autonomous IoT command system. Dynamic behavior is instead achieved by treating new reports, detection results and rescue updates as state changes.



End-to-End Operational Example

Suppose a flood affects Zone C. A drone video produces observations of several people and a submerged vehicle. A citizen reports that a nearby road is blocked.

The database now contains people/vehicle observations, a reported road hazard and the current resource inventory. The zone priority increases because the affected situation has become more urgent.

The coordination engine identifies an available suitable resource and a volunteer whose profile matches the task. A candidate route is generated and checked against the reported blocked road.

The recommendation is displayed on the command dashboard. The authority can approve or modify it. After rescue, the person is checked into a shelter and the status becomes available for authorized family lookup.

If a new road report arrives later, the route can be evaluated again rather than assuming that the previous route remains safe.

5. Mathematical Foundations & Quantitative Frameworks

The MVP uses transparent scoring models so that recommendations can be explained to a human operator. These equations represent the proposed framework and should be calibrated against actual implementation and test scenarios.

5.1 Zone Priority Model

For zone z , define a normalized priority score:

$$P_z = w_A A_z + w_C C_z + w_S S_z + w_H H_z$$

A is the affected-population measure. C represents critical cases. S represents SOS/emergency requests. H represents hazard severity. The weights determine the relative importance of each factor.

5.2 Resource Allocation Model

For resource r considered for zone z :

$$Q_{r,z} = w_C C_{r,z} + w_S S_{r,z} + w_A A_r - w_D D_{r,z} - w_T T_{r,z}$$

C represents capability suitability, S represents skill suitability, A represents availability, D represents distance and T represents estimated travel time.

Unavailable or incompatible resources should be filtered before scoring so that an unsuitable resource cannot be selected merely because it is nearby.

5.3 Volunteer Matching Model

For volunteer v and zone z :

$$V_{v,z} = w_K K_{v,z} + w_A A_v - w_D D_{v,z}$$

K represents skill/profile compatibility. A represents availability and D represents distance.

5.4 Route Hazard Model

For candidate route R , a hazard penalty can be represented as:

$$H(R) = \sum \lambda_i I_i(R)$$

I indicates whether route R intersects hazard i . λ represents the severity penalty of that hazard.

A route may be rejected or flagged if its accumulated hazard penalty exceeds a configured threshold.

5.5 Mathematical Summary Matrix

Metric	Meaning	Decision Use
P_z	Zone priority score	Rank affected zones.
$Q_{r,z}$	Resource suitability	Rank available resources.
$V_{v,z}$	Volunteer suitability	Rank suitable volunteers.
$H(R)$	Route hazard penalty	Reject or flag unsafe routes.
$Os = Ns / Cs$	Shelter occupancy ratio	Monitor shelter load.

6. Database Schema & Security Infrastructure

6.1 Entity-Relationship Overview



The database is location-centric. Entities that influence operational decisions should carry a geographic representation or reference a geographic entity.

6.2 Database Tables

Table	Important Fields	Purpose
users	id, role, created_at	Identity and role.
citizen_profiles	user_id, profile fields	Citizen information.
zones	id, name, severity, priority, geometry	Operational flood zones.
hazards	id, zone_id, type, severity, geometry	Road/flood/infrastructure hazards.
reports	id, user_id, type, severity, location, status	Community observations.
sos_requests	id, user_id, location, severity, status	Emergency requests.
drone_observations	id, class, confidence, location, timestamp	Computer-vision observations.
resources	id, type, capability, availability, location	Rescue resources.
volunteer_profiles	user_id, skills, availability, location	Volunteer capabilities.
resource_assignments	id, zone_id, resource_id, volunteer_id, status	Recommended/approved assignments.
shelters	id, name, capacity, occupancy, location	Shelter information.
shelter_checkins	id, person_id, shelter_id, status, timestamp	Rescue and shelter status.

6.3 Security & Row-Level Access

Access control should ensure that citizens cannot access authority-only operational information. Volunteers should only access assignments and profile information appropriate to their role. Shelter operators should only manage records within their permitted shelter scope.

Family-facing status lookup should expose only intentionally shareable information such as a person's safe/check-in status. Internal operational information, resource locations and authority notes should remain protected.

Privileged service credentials must never be exposed in the browser. Secrets should be stored through environment variables and deployment secret management.

7. Local Development & Installation Guide

7.1 Prerequisites

Component	Requirement
Node.js	Maintained LTS release compatible with the selected frontend/backend packages.
npm	Bundled with the selected Node.js release.
Python	3.10+ recommended for YOLO inference.
Git	Current stable release.
Browser	Modern browser; WebXR support depends on device/browser.
Mapbox	Access token for map functionality.
Supabase	Project URL and public client key.

7.2 Setup

```
git clone <repository-url>
cd flood-resilience

npm install

python -m venv .venv

# Windows
.venv\Scripts\activate

# macOS/Linux
source .venv/bin/activate

pip install -r ai/requirements.txt
```

The exact repository commands should be updated once the actual implementation repository is finalized.

7.3 Environment Variables

Variable	Required	Purpose
VITE_MAPBOX_TOKEN	Yes	Mapbox access for the frontend.
SUPABASE_URL	Yes	Supabase project endpoint.
SUPABASE_ANON_KEY	Yes	Public client key.
SUPABASE_SERVICE_ROLE_KEY	Backend only	Privileged operations. Never expose to browser.
API_BASE_URL	Yes	Backend API address.
YOLO_MODEL_PATH	AI service	Selected YOLO model path.

7.4 Running the Development Suite

```
# Frontend
cd frontend
npm run dev

# Backend
cd backend
npm run dev

# AI service
```



```
source .venv/bin/activate  
python ai/detection/inference.py
```

Credentials must not be committed to Git. Local environment files and deployment secret stores should be used.

8. Verification, Testing & Quality Assurance

8.1 Static Verification

```
npm run lint
npm run typecheck
npm run build
npm test
```

The exact commands depend on the final package configuration. They should be included in the repository's package scripts.

8.2 Feature Validation

Area	Test	Expected Result
WebXR	Scenario water level changes	Scenario state changes consistently.
YOLO	Frame contains people	People detections are produced.
YOLO	Frame contains vehicle	Vehicle may be flagged.
Community	Blocked road report	Geospatial hazard is stored.
Priority	Higher SOS zone	Priority increases appropriately.
Allocation	Nearest resource unavailable	Unavailable resource is excluded.
Allocation	More capable distant resource	Suitability can favor capability.
Routing	Route intersects hazard	Route is flagged/rejected.
Shelter	Person checks in	Safe status and occupancy update.

8.3 Edge Cases & Operational Guardrails

Failure State	Guardrail
No resource available	Show no-assignment condition rather than fabricate a dispatch.
No safe route	Flag route unavailable and require authority intervention.
Low-confidence YOLO result	Treat as an observation requiring review.
Conflicting citizen reports	Retain source and verification state.
Offline report	Store pending report locally and synchronize later.
Shelter full	Prevent normal assignment to full shelter and surface alternatives.
Duplicate SOS	Detect or merge duplicate requests using request/user/time rules.
Stale hazard	Record timestamp and surface data age to the operator.

Safety principle: the platform should fail visibly. When information is missing or contradictory, it should surface uncertainty rather than silently making an unsafe operational decision.

9. Multi-Platform Deployment & DevOps Pipeline

9.1 Web Deployment

The frontend can be built as a modern web application and deployed through a static web hosting platform. Public client configuration can be provided through environment variables.

```
npm install
npm run build
```

The backend and AI inference components should remain separate from static frontend assets.

9.2 Containerized Deployment

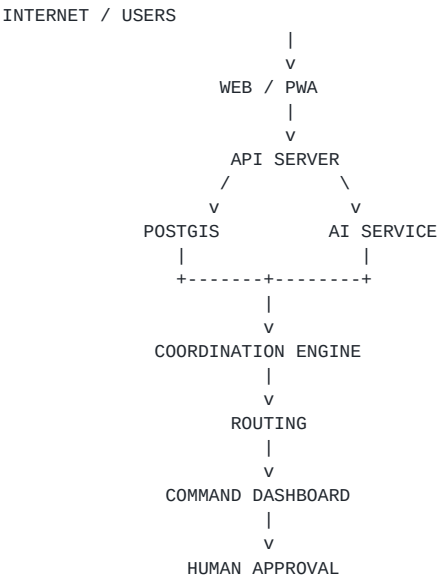
Docker can provide a reproducible runtime for the backend and AI processing components.

```
docker build -t flood-resilience-backend .
docker run --env-file .env -p 3000:3000 flood-resilience-backend
```

9.3 Optional Mobile Packaging

The first implementation target is web/PWA. If the project later requires a native Android package, Capacitor can be evaluated. It should not become a core dependency unless the submission requires native mobile packaging.

Deployment Flow



The deployment architecture keeps operational decision logic separate from client presentation and AI inference.

10. Changelog, Roadmap, Troubleshooting & License

10.1 Development History

Phase	Design Decision
Architecture Scoping	Reduced the project from disconnected technology features to a single disaster-response workflow.
WebXR Scoping	Moved from native AR/VR ambitions toward browser-based WebXR for feasibility.
Drone Scoping	Use simulated/pre-recorded drone footage for prototyping.
Computer Vision Scope	Use YOLO primarily for people and vehicles instead of claiming reliable property-damage detection.
Community Intelligence	Use citizen reports for blocked roads and infrastructure/property damage information.
Coordination	Defined zone-based resource allocation and volunteer matching as the core response feature.
Routing	Use route generation plus hazard checks rather than autonomous navigation.
Shelter Tracking	Add check-in status so separated families can determine whether a person reached safety.

10.2 Roadmap

Milestone	Objective
M1	Citizen reporting and geospatial dashboard.
M2	WebXR flood scenario.
M3	Drone video prototype and YOLO detection.
M4	Zone priority, resource allocation and volunteer matching.
M5	Hazard-aware route validation and human approval.
M6	Shelter check-in and family status.
M7	Integrated testing and deployment.
Future	Optional IoT sensors, richer remote sensing, stronger offline support and native mobile packaging.

10.3 Troubleshooting Matrix

Problem	Likely Cause	Resolution
Map does not load	Missing/invalid Mapbox token	Check environment variable and allowed origins.
Report not stored	API/database configuration	Check validation, credentials and schema.
YOLO finds nothing	Poor footage or model mismatch	Inspect frames, target classes and confidence threshold.
Route crosses hazard	Hazard check missing	Run spatial intersection before approval.
WebXR unavailable	Browser/device limitation	Provide a normal browser simulation fallback.
Offline report lost	Local queue failure	Persist pending report in IndexedDB.

10.4 Licensing & Intellectual Property

The final repository license should be selected by the project team. This proposal does not assert a license that has not yet been selected.

If an open-source license such as MIT is selected, the final repository should contain the complete license text and appropriate third-party dependency notices.

10.5 Error Log Book

Challenge	Cause / Limitation	Design Response
Native AR/VR expertise unavailable	Team skill and time constraints	Use browser-based WebXR.
YOLO does not reliably detect property damage	Model/task mismatch	Limit CV to people/vehicles and use community reports.
Road damage difficult to infer	Aerial imagery may not provide reliable classification	Allow people to report blocked/damaged roads.
Architecture became too broad	Too many technologies introduced	Keep the MVP centered around coordination.
Autonomous dispatch risk	AI observation is not operational authority	Keep human approval before dispatch.

Appendix A — Core Technology Reference

Technology	Project Role	MVP Status
React	Web application and dashboard	Core
Mapbox GL JS	Geospatial visualization	Core
A-Frame / Three.js	WebXR simulation	Core
WebXR	Immersive browser simulation	Core
PWA	Installable web experience	Core
IndexedDB	Offline/pending reports	Fallback
Node.js / Express	API and business logic	Core
Python / YOLOv8	Drone computer vision	Prototype
Supabase	Backend platform	Core
PostgreSQL	Relational database	Core
PostGIS	Geospatial data	Core
Mapbox Directions	Route generation	Core
Turf.js	Spatial route/hazard checks	Core
Docker	Deployment packaging	Support

Appendix B — Core Coordination Pseudocode

```
function recommendResponse(zone, resources, volunteers):

    priority = scoreZone(zone)

    eligibleResources = filter(
        resources,
        resource =>
            resource.available
            AND capabilityMatches(resource, zone)
    )

    eligibleVolunteers = filter(
        volunteers,
        volunteer =>
            volunteer.available
            AND skillMatches(volunteer, zone)
    )

    rankedResources = sortByScore(
        eligibleResources,
        suitabilityScore(resource, zone)
    )

    rankedVolunteers = sortByScore(
        eligibleVolunteers,
        volunteerScore(volunteer, zone)
    )

    recommendation = {
        zonePriority: priority,
        resource: first(rankedResources),
        volunteer: first(rankedVolunteers)
    }

    return recommendation
```

The recommendation is then passed to the command dashboard. The authority can approve, modify or reject it before a rescue action is treated as authorized.

Appendix C — Submission Integrity

This document is a technical proposal and MVP specification. Technology choices, mathematical models and architecture described as proposed should be synchronized with the actual implementation as development progresses.

The document intentionally avoids claiming that every component is already production-ready. In particular, YOLO is not claimed to solve property damage detection, routing is not autonomous navigation, and the coordination engine is not an autonomous rescue authority.

The strongest technical claim is therefore the integration workflow: preparedness through WebXR, situation awareness through drone and community inputs, coordination through zone/resource/volunteer scoring, route validation, human approval and shelter-based status tracking.

End of Technical Submission — Version 0.1

Appendix D — Executable Database DDL & RLS Reference

This appendix provides a reference PostgreSQL/PostGIS migration for the proposed application schema. It is not presented as an already-deployed database; it becomes the implementation baseline when the application DB is created.

D.1 Complete PostgreSQL DDL

```
create extension if not exists postgis;

create table if not exists users (
    id uuid primary key references auth.users(id) on delete cascade,
    role text not null default 'citizen'
    check (role in ('citizen','volunteer','authority','shelter_operator')),
    created_at timestamptz not null default now()
);

create table if not exists zones (
    id bigserial primary key,
    name text not null,
    severity numeric(5,2) not null default 0 check (severity between 0 and 100),
    priority numeric(8,4) not null default 0,
    population integer not null default 0 check (population >= 0),
    vulnerable_population integer not null default 0 check (vulnerable_population >= 0),
    active_sos integer not null default 0 check (active_sos >= 0),
    geometry geometry(Polygon,4326),
    created_at timestamptz not null default now(),
    updated_at timestamptz not null default now()
);

create table if not exists hazards (
    id bigserial primary key,
    zone_id bigint references zones(id) on delete set null,
    type text not null,
    severity numeric(5,2) not null default 0 check (severity between 0 and 100),
    description text,
    geometry geometry(Geometry,4326),
    reported_at timestamptz not null default now(),
    expires_at timestamptz
);

create table if not exists reports (
    id bigserial primary key,
    user_id uuid not null references users(id) on delete cascade,
    zone_id bigint references zones(id) on delete set null,
    type text not null,
    severity numeric(5,2) not null default 0 check (severity between 0 and 100),
    description text,
    location geometry(Point,4326),
    status text not null default 'pending'
```

```

        check (status in ('pending','verified','rejected','resolved')),
        created_at timestamptz not null default now()
    );

create table if not exists resources (
    id bigserial primary key,
    type text not null,
    capability text,
    availability text not null default 'available'
        check (availability in ('available','assigned','unavailable')),
    quantity integer not null default 1 check (quantity > 0),
    location geometry(Point,4326),
    updated_at timestamptz not null default now()
);

create table if not exists volunteer_profiles (
    user_id uuid primary key references users(id) on delete cascade,
    skills text[] not null default '{}',
    availability text not null default 'available',
    location geometry(Point,4326),
    equipment text[] not null default '{}'
);

create table if not exists shelters (
    id bigserial primary key,
    name text not null,
    capacity integer not null check (capacity >= 0),
    occupancy integer not null default 0 check (occupancy >= 0 and occupancy <= capacity),
    location geometry(Point,4326) not null,
    created_at timestamptz not null default now()
);

create table if not exists resource_assignments (
    id bigserial primary key,
    zone_id bigint not null references zones(id) on delete cascade,
    resource_id bigint references resources(id) on delete set null,
    volunteer_id uuid references volunteer_profiles(user_id) on delete set null,
    status text not null default 'recommended'
        check (status in ('recommended','approved','modified','rejected','completed')),
    created_by uuid references users(id) on delete set null,
    created_at timestamptz not null default now()
);

create table if not exists shelter_checkins (
    id bigserial primary key,
    person_id uuid not null references users(id) on delete cascade,
    shelter_id bigint not null references shelters(id) on delete cascade,
    status text not null default 'checked_in',
    checked_in_at timestamptz not null default now(),
    checked_out_at timestamptz
);

create index if not exists zones_geometry_gix on zones using gist (geometry);
create index if not exists hazards_geometry_gix on hazards using gist (geometry);
create index if not exists reports_location_gix on reports using gist (location);
create index if not exists resources_location_gix on resources using gist (location);
create index if not exists shelters_location_gix on shelters using gist (location);

```

D.2 Row-Level Security Policies

RLS protects user-owned information and separates citizen, volunteer, authority and shelter-operator access. These policies are reference implementation examples and must be tested against the final schema.

```

alter table reports enable row level security;
alter table volunteer_profiles enable row level security;
alter table resource_assignments enable row level security;
alter table shelter_checkins enable row level security;

create policy report_owner_insert
on reports for insert to authenticated
with check (user_id = auth.uid());

create policy report_owner_select
on reports for select
using (
    user_id = auth.uid()
    or exists (
        select 1 from users u

```



```

        where u.id = auth.uid() and u.role = 'authority'
    )
);

create policy volunteer_owner_select
on volunteer_profiles for select
using (
    user_id = auth.uid()
    or exists (
        select 1 from users u
        where u.id = auth.uid() and u.role = 'authority'
    )
);

create policy volunteer_owner_update
on volunteer_profiles for update
using (user_id = auth.uid())
with check (user_id = auth.uid());

create policy assignment_authority_insert
on resource_assignments for insert to authenticated
with check (
    exists (
        select 1 from users u
        where u.id = auth.uid() and u.role = 'authority'
    )
);

create policy assignment_authority_or_volunteer_select
on resource_assignments for select
using (
    volunteer_id = auth.uid()
    or exists (
        select 1 from users u
        where u.id = auth.uid() and u.role = 'authority'
    )
);

create policy shelter_checkin_authorized_select
on shelter_checkins for select
using (
    person_id = auth.uid()
    or exists (
        select 1 from users u
        where u.id = auth.uid()
        and u.role in ('authority','shelter_operator')
    )
);

```

D.3 Explicit LaTeX Formula Reference

Zone priority:

$$P_z = w_A A_z + w_C C_z + w_S S_z + w_H H_z$$

Resource suitability:

$$Q_{\{r,z\}} = w_C C_{\{r,z\}} + w_S S_{\{r,z\}} + w_A A_r - w_D D_{\{r,z\}} - w_T T_{\{r,z\}}$$

Volunteer suitability:

$$V_{\{v,z\}} = w_K K_{\{v,z\}} + w_A A_v - w_D D_{\{v,z\}}$$

Route hazard penalty:

$$H(R) = \sum_{i=1}^n \lambda_i I_i(R)$$

Shelter occupancy:

$$O_s = \frac{N_s}{C_s}$$

Variable	Definition
A _z	Affected population measure for zone z.
C _z	Critical/vulnerable case measure.
S _z	SOS/emergency request measure.
H _z	Hazard severity measure.
w _*	Non-negative calibration weights.
D	Distance penalty.
T	Estimated travel-time penalty.
I _i (R)	1 when route R intersects hazard i; otherwise 0.
lambda _i	Severity penalty assigned to hazard i.
N _s / C _s	Shelter occupancy divided by shelter capacity.

Appendix E — Deployment Configuration Reference

E.1 Backend Dockerfile

```
FROM node:20-alpine
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
RUN npm run build
EXPOSE 3000
CMD ["npm", "start"]
```

E.2 Deployment Sequence

```
npm ci
npm run lint
npm run typecheck
npm test
npm run build

docker build -t flood-resilience-backend ./backend
docker run --rm --env-file .env -p 3000:3000 flood-resilience-backend
```

Appendix F — Environment & Secret Reference

Variable	Required	Acquisition / handling
VITE_MAPBOX_TOKEN	Yes	Create Mapbox token; restrict allowed origins.
SUPABASE_URL	Yes	Copy project URL from Supabase settings.
SUPABASE_ANON_KEY	Yes	Copy public client key from Supabase API settings.
SUPABASE_SERVICE_ROLE_KEY	Backend only	Copy server secret; never expose in frontend.
API_BASE_URL	Yes	Set to local or deployed API address.
YOLO_MODEL_PATH	AI service	Path to the selected YOLO model file.

Appendix G — License Text

Use the following MIT text only if the repository LICENSE file is actually the MIT License. If a different license was selected, replace this appendix with the exact repository license.

MIT License

Copyright (c) 2026 Flood Resilience & Response Platform Project Team

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.

Appendix H — Submission Verification Matrix

Requirement	Coverage in this document
Numbered TOC	Sections 1–10 plus appendices.
ASCII architecture	Section 2 data-flow diagrams.
Repository structure	Section 3 and current GitHub snapshot.
Feature workflows	Section 4 with input, processing and output.
Mathematical formulations	Section 5 plus Appendix D.3.
SQL / DDL	Appendix D.1.
RLS	Appendix D.2.
Environment variables	Section 7 and Appendix F.
Testing and guardrails	Section 8.
Deployment	Section 9 and Appendix E.
Changelog / roadmap	Section 10.
Troubleshooting / error log	Section 10.
License	Section 10 and Appendix G, conditional on repository license.